

Parameter injection（参数注入）全解析

这段内容讲解了 NiceGUI 基于 FastAPI 实现的**参数注入（Parameter Injection）**特性——页面函数可通过声明参数，自动从 URL 中提取**路径参数**、**查询参数**，甚至直接获取完整的请求对象（用于访问请求体、请求头、Cookie 等），无需手动解析 URL 或请求数据，大幅简化参数处理逻辑。

核心背景：FastAPI 底层支撑

NiceGUI 内置 FastAPI 作为 Web 后端框架，因此直接继承了 FastAPI「声明式参数解析」的核心优势：

- 开发者只需在页面函数中声明参数（含类型提示），框架会自动完成：
 1. 从 URL 路径中提取「路径参数」（如 `/icon/{icon}` 中的 `icon`）；
 2. 从 URL 查询字符串中提取「查询参数」（如 `?amount=5` 中的 `amount`）；
 3. 自动类型转换（如把字符串 `amount=5` 转为整数 `5`）；
 4. 处理默认值（未传递参数时使用默认值）；
 5. 自动校验参数（如传递非整数的 `amount` 会返回 422 错误）。

示例代码逐行拆解

```
from nicegui import ui

# 定义带路径参数的页面路由: /icon/{icon}
@ui.page('/icon/{icon}')
def icons(icon: str, amount: int = 1):
    # 1. 路径参数 icon: 从 URL 路径 /icon/{icon} 中提取（必传）
    #    类型提示 str: 框架自动将提取的字符串赋值给 icon 参数
    ui.label(icon).classes('text-h3') # 显示图标名称

    # 2. 查询参数 amount: 从 URL 查询字符串 ?amount=xxx 提取（可选，默认值 1）
    #    类型提示 int: 框架自动将查询参数的字符串转为整数
    with ui.row(): # 横向排列图标
        # 根据 amount 数量循环生成图标组件
        [ui.icon(icon).classes('text-h3') for _ in range(amount)]

# 根路径页面: 提供跳转链接（演示不同参数传递方式）
@ui.page('/')
def page():
    # 链接1: 路径参数 star + 查询参数 amount=5 → /icon/star?amount=5
    ui.link('Star', '/icon/star?amount=5')
    # 链接2: 仅路径参数 home → 查询参数 amount 使用默认值 1 → /icon/home
    ui.link('Home', '/icon/home')
    # 链接3: 路径参数 water_drop + 查询参数 amount=3 → /icon/water_drop?amount=3
    ui.link('Water', '/icon/water_drop?amount=3')

ui.run()
```

关键特性详解

1. 路径参数 (Path Parameters)

- **定义方式**: 在 `@ui.page` 的路由字符串中用 `{参数名}` 声明 (如 `/icon/{icon}`) ;
- **必传性**: 路径参数默认是必填项, 若访问 `/icon/` (无 `icon` 参数) 会返回 404 错误;
- **类型提示**: 页面函数中声明参数类型 (如 `icon: str`) , 框架会自动校验类型 (若路径参数无法转为声明类型, 返回 422 错误) 。

2. 查询参数 (Query Parameters)

- **定义方式**: 页面函数中声明「非路径参数」+ 可选默认值 (如 `amount: int = 1`) ;
- **传递方式**: URL 中用 `?参数名=值` 传递, 多个参数用 `&` 分隔 (如 `/icon/star?amount=5&color=red`) ;
- **可选性**: 若声明了默认值则为可选参数 (未传递时用默认值) ; 若未声明默认值 (如 `amount: int`) , 则为必填查询参数 (未传递会返回 422 错误) ;
- **类型转换**: 框架自动将查询参数的字符串转为声明的类型 (如 `amount=5` → `int 5` , `is_show=true` → `bool True`) 。

3. 访问完整请求对象

若需要更复杂的请求信息 (如请求头、Cookie、请求体) , 可在页面函数中声明 `request: Request` 参数 (需导入 FastAPI 的 `Request` 类) , 直接获取完整的请求对象:

```
from fastapi import Request
from nicegui import ui

@ui.page('/request-info')
def show_request_info(request: Request):
    # 获取请求头
    ui.label(f'请求头 User-Agent: {request.headers.get("user-agent")}')
    # 获取 Cookie
    ui.label(f'Cookie: {request.cookies}')
    # 获取客户端 IP
    ui.label(f'客户端 IP: {request.client.host}')

ui.run()
```

4. 参数校验与错误处理

FastAPI 内置的参数校验规则会自动生效:

- 若传递的查询参数无法转为声明类型 (如 `/icon/star?amount=abc`) , 框架会返回 422 `Unprocessable Entity` 错误, 并提示 `[value is not a valid integer]` ;
- 若缺少必填参数 (如 `/icon/star` 但 `amount` 声明为 `amount: int` 无默认值) , 会返回 422 错误, 提示 `[field required]` ;
- 可通过 NiceGUI 自定义错误页面, 优化用户体验 (如捕获 422 错误并显示友好提示) 。

运行效果说明

启动脚本后访问 `http://localhost:8080`（默认端口）：

1. 点击「Star」链接 → 跳转到 `/icon/star?amount=5`：显示「star」标签 + 5 个 star 图标；
2. 点击「Home」链接 → 跳转到 `/icon/home`：显示「home」标签 + 1 个 home 图标（amount 用默认值 1）；
3. 点击「Water」链接 → 跳转到 `/icon/water_drop?amount=3`：显示「water_drop」标签 + 3 个 water_drop 图标。

适用场景

- 动态生成页面内容（如根据商品 ID 显示商品详情：`/product/{product_id}`）；
- 接收用户通过 URL 传递的配置（如每页显示数量、筛选条件）；
- 访问请求元数据（如验证请求头中的 Token、获取客户端信息）。

注意事项

1. 路径参数和查询参数的命名不能冲突（如不能同时声明 `def icons(icon: str, icon: int)`）；
2. 类型提示仅支持 FastAPI 兼容的类型（int/float/bool/str 等基础类型，以及 Pydantic 模型）；
3. 若需传递复杂参数（如 JSON），建议用 POST 请求 + 请求体，而非查询参数（查询参数长度有限且易暴露）。